Avoid Nested Locks

The first idea is the simplest: don't acquire a lock if you already hold one. If you stick to this guideline, it's impossible to get a deadlock from the lock usage alone because each thread only ever holds a single lock. You could still get deadlock from other things (like the threads waiting for each other), but mutex locks are probably the most common cause of deadlock. If you need to acquire multiple locks, do it as a single action with `std::lock` in order to acquire them without deadlock.

Avoid Calling User-Supplied Code While Holding a Lock

This is a simple follow-on from the previous guideline. Because the code is user supplied, you have no idea what it could do; it could do anything, including acquiring a lock. If you call user-supplied code while holding a lock, and that code acquires a lock, you've violated the guideline on avoiding nested locks and could get deadlock. Sometimes this is unavoidable; if you're writing generic code such as the stack in section 3.2.3, every operation on the parameter type or types is user-supplied code. In this case, you need a new guideline.

Acquire Locks in a Fixed Order

If you absolutely must acquire two or more locks, and you can't acquire them as a single operation with `std::lock`, the next-best thing is to acquire them in the same order in every thread. I touched on this in section 3.2.4 as one way of avoiding deadlock when acquiring two mutexes: the key is to define the order in a way that's consistent between threads. In some cases, this is relatively easy. For example, look at the stack from section 3.2.3—the mutex is internal to each stack instance, but the operations on the data items stored in a stack require calling user-supplied code. You can, however, add the constraint that none of the operations on the data items stored in the stack should perform any operation on the stack itself. This puts the burden on the user of the stack, but it's rather uncommon for the data stored in a container to access that container, and it's quite apparent when this is happening, so it's not a particularly difficult burden to carry.